

PREGUNTAS TEÓRICAS

CheckPoint 4

1. ¿Cuál es la diferencia entre una lista y una tupla en Python?

Mutabilidad y Estructura Interna

- **Listas (list):** Son arreglos dinámicos mutables. Permiten operaciones que alteran su tamaño e integridad en tiempo de ejecución (append, pop, insert).
- **Tuplas (tuple):** Son arreglos estáticos inmutables. Una vez reservado su espacio en memoria, no se puede alterar su contenido ni su longitud sin generar un nuevo objeto (p. 1). [\[1, 2, 3\]](#)

Gestión de Memoria y Rendimiento

- **Listas:** Implementan un mecanismo de **sobre-asignación (over-allocation)**. Python reserva espacios extra preventivos en memoria para garantizar que `.append()` funcione en tiempo amortizado $\mathcal{O}(1)$. Esto genera un desperdicio leve de memoria (*overhead*). [\[1, 2\]](#)
- **Tuplas:** Al tener tamaño fijo garantizado, Python calcula el tamaño exacto del bloque de memoria necesario de manera contigua. Son considerablemente más rápidas en tareas de iteración y acceso directo. [\[1, 2, 3\]](#)

```
python
import sys
```

```
# Comparación directa de peso en memoria (Bytes)
```

```
lista_vacia = []
```

```
tupla_vacia = ()
```

```
print(sys.getsizeof(lista_vacia)) # ~56 bytes (incluye overhead de
redimensionamiento)
```

```
print(sys.getsizeof(tupla_vacia)) # ~40 bytes (estructura mínima limpia)
```

Casos de Uso

- **Listas:** Colecciones homogéneas de datos destinados a variar, como un carrito de compras dinámico.
- **Tuplas:** Registros de datos heterogéneos con significado posicional fijo, como coordenadas geográficas (latitud, longitud) o configuraciones de base de datos imperturbables. [\[1\]](#)

2. ¿Cuál es el orden de las operaciones?

Python procesa las expresiones numéricas implementando la jerarquía algebraica matemática estricta conocida bajo el acrónimo **PEMDAS** (p. 1). Cuando existen operadores con la misma jerarquía dentro de una línea, el motor los resuelve bajo la regla de **asociatividad de izquierda a derecha** (salvo la potenciación, que se evalúa de derecha a izquierda) (p. 1).

Jerarquía Estricta (De Mayor a Menor Prioridad)

1. Paréntesis () (p. 1)
2. Exponentes o potencias ** (p. 1)
3. Multiplicación *, División /, División Entera //, Módulo o residuo % (p. 1)
4. Adición (Suma) +, Sustracción (Resta) - (p. 1)

Ejemplo Práctico Desglosado

```
python
resultado = 5 + 3 * 2 ** 3 / (4 - 2)
# Paso 1 (Paréntesis): 5 + 3 * 2 ** 3 / 2
# Paso 2 (Exponente): 5 + 3 * 8 / 2
# Paso 3 (Mult/Div de izq a der): 3 * 8 = 24 -> 24 / 2 = 12
# Paso 4 (Suma): 5 + 12 = 17.0
print(resultado) # Output: 17.0
```

3. ¿Qué es un diccionario Python?

Un diccionario es una colección mapeada **mutable** que enlaza pares únicos de clave: valor (p. 1). A diferencia de los índices correlativos numéricos de las listas, un diccionario accede a sus datos mediante claves (p. 1).

Implementación Interna (Tablas Hash Compactas)

Desde la versión de Python 3.6+ (y garantizado en la 3.7+), los diccionarios utilizan una estructura optimizada por Raymond Hettinger, el cual propuso una idea revolucionaria inspirada en **separar los índices de los datos reales** creando dos arreglos internos en lugar de uno: un índice denso de *hashes* compacto y un arreglo secundario que preserva explícitamente el **orden de inserción**.

Las claves deben ser de un tipo **hashable e inmutable** (como strings, enteros o tuplas), asegurando búsquedas inmediatas en tiempo constante promedio de $O(1)$. [[1](#), [2](#), [3](#), [4](#), [5](#)]

```
python
# Ejemplo de uso práctico y simulación de base de datos rápida
usuario = {
    "id": 105,
    "nombre": "Carlos",
    "roles": ["admin", "editor"],
    "activo": True
}
```

```
# Acceso inmediato  $O(1)$  directo por clave
print(usuario["nombre"]) # Output: Carlos
```

4. ¿Cuál es la diferencia entre el método .sort() y la función sorted()?

Ambos mecanismos ordenan colecciones numéricas o alfabéticas bajo el algoritmo interno **Timsort**, pero difieren drásticamente en cómo afectan al estado del objeto original y en qué tipos de datos pueden procesar (p. 1):

Característica	Método .sort()	Función sorted()
Mutación	In-place: Modifica directamente el objeto original. Ahorra memoria al no duplicar datos (p. 1).	Pure function: Preserva el iterable original intacto. Devuelve una copia nueva (p. 1).
Retorno	Devuelve explícitamente None (p. 1).	Devuelve una nueva list con los elementos ordenados (p. 1).
Compatibilidad	Exclusivo de objetos tipo lista (list) (p. 1).	Universal. Admite cualquier iterable (listas, tuplas, diccionarios, sets) (p. 1).

```
python
# Demostración del comportamiento
edades_lista = [25, 18, 40]
edades_tupla = (31, 12, 22)

# Uso de .sort()
edades_lista.sort()
print(edades_lista) # [18, 25, 40] (Lista mutada)

# Uso de sorted() en una tupla inmutable
nueva_lista = sorted(edades_tupla)
print(edades_tupla) # (31, 12, 22) (Tupla original intacta)
print(nueva_lista) # [12, 22, 31] (Nueva lista creada)
```

5. ¿Qué es un operador de reasignación?

Un **operador de asignación compuesta o reasignación** actualiza directamente el valor de una variable tomando como base el estado inmediato anterior de esa misma variable, todo bajo una única instrucción compacta (p. 1). Combina un operador binario aritmético o lógico junto al operador estándar = (p. 1).

Tabla de Equivalencias Críticas

- $x += y$ es el azúcar sintáctico de $x = x + y$ (p. 1)
- $x -= y$ es el azúcar sintáctico de $x = x - y$
- $x *= y$ es el azúcar sintáctico de $x = x * y$
- $x /= y$ es el azúcar sintáctico de $x = x / y$

El Secreto Interno de los Operadores en Mutables vs Inmutables

Cuando aplicamos un operador de reasignación, Python intenta invocar preferentemente los métodos mágicos *in-place* correspondientes (como `__iadd__`).

- En objetos **mutables** (como una lista), `lista += [4]` modifica la lista directamente en el mismo espacio físico de memoria.
- En objetos **inmutables** (como enteros, cadenas o tuplas), `tupla += (4,)` no modifica la tupla; destruye el puntero original, genera una tupla nueva combinada en otra dirección de memoria y reasigna la variable a esta dirección.

python

Ejemplo con inmutables (reasignación de dirección)

puntos = 10

print(id(puntos)) # Dirección A

puntos += 5 # Operador de reasignación por suma

print(id(puntos)) # Dirección B (Nueva dirección asignada a la variable)